

SESIÓN TP9

Sistemas Operativos 1

Mayo 2026

1 INTRODUCCIÓN

Los punteros son herramientas fundamentales en la programación, proporcionando acceso directo a la memoria de una computadora. Su comprensión y dominio son esenciales para cualquier programador que busque optimizar la eficiencia y la flexibilidad de sus programas.

Esta sesión se centra en explorar los conceptos fundamentales de los punteros en el lenguaje de programación C. Desde la manipulación de datos hasta la gestión de la memoria dinámica, se verá cómo los punteros permiten a los programadores realizar tareas avanzadas con precisión y eficiencia. A través de diversos ejercicios prácticos, se intenta fortalecer la comprensión y habilidades en el uso de esta herramienta de programación.

2 PUNTEROS EN C

Un puntero es una variable que almacena la dirección de memoria de otra variable. Es decir, en lugar de contener el valor de una variable, un puntero contiene la dirección de memoria donde se encuentra almacenado el valor de esa variable.

Entre las principales aplicaciones de los punteros se encuentran:

- **Manipulación de estructuras de datos complejas:** los punteros permiten acceder y manipular directamente la memoria donde se almacenan las estructuras de datos, lo que puede resultar más eficiente que copiar y manipular los datos directamente.
- **Asignación dinámica de memoria:** los punteros permiten asignar y liberar memoria de forma dinámica durante la ejecución del programa, lo que puede ser útil para construir estructuras de datos complejas o para evitar problemas de fragmentación de memoria.
- **Paso de argumentos:** los punteros se utilizan comúnmente para pasar argumentos a funciones de forma eficiente, especialmente cuando se trabaja con estructuras de datos grandes o complejas.

Es habitual utilizar punteros de diferentes tipos, como por ejemplo:

```
int    *ip;    /* pointer to an integer */
double *dp;    /* pointer to a double */
float  *fp;    /* pointer to a float */
char   *ch     /* pointer to a character */
```

La variable `ip`, por ejemplo, es una variable que apunta a una dirección de memoria en la que se almacena el dato entero (integer).

3 EJEMPLOS A REALIZAR

A continuación se muestran 5 ejemplos de punteros en C para realizar durante la sesión teórico-práctica, que se han extraído de la página web https://www.tutorialspoint.com/cprogramming/c_pointers.htm

Ejercicio 1

El siguiente código muestra cómo podemos **modificar el valor de un entero mediante un puntero**.

```
int main () {
    int  var = 20;    /* actual variable declaration */
    int  *ip;         /* pointer variable declaration */

    ip = &var; /* store address of var in pointer variable*/

    printf("Address of var variable: %x\n", &var);
    printf("Address stored in ip variable: %x\n", ip);
    printf("Value of *ip variable: %d\n", *ip);

    *ip = 30; /* Change value of the contents of *ip */
    printf("Value of variable var: %d\n", var);

    return 0;
}
```

Los punteros tienen múltiples conceptos asociados que son muy importantes en el lenguaje C. El hecho de entenderlos permite realizar determinadas operaciones de una manera más eficiente.

Ejercicio 2

Una de las operaciones más habituales donde se utilizan punteros son las **manipulaciones de vectores**. Aquí veremos un ejemplo que muestra este concepto.

```
const int MAX = 3;

int main () {

    int  var[] = {10, 100, 200};
    int  i, *ptr;

    /* let us have array address in pointer */
    ptr = var;

    for ( i = 0; i < MAX; i++) {

        printf("Address of var[%d] = %x\n", i, &var[i]);
        printf("Value of ptr for iteration %d is %d\n", i, ptr);

        /* move to the next location */
        ptr++;
    }

    return 0;
}
```

Este ejemplo muestra uno de los conceptos más interesantes en la manipulación de vectores: podemos acceder a los valores de un vector utilizando punteros (`ptr`) y no como variable original

(`var`). Lo podemos hacer ya que se pueden realizar operaciones aritméticas (`++`, `--`, `+`, `-`) con los punteros.

1. ¿Qué hace `ptr++`? ¿En cuánto se incrementa el valor de `ptr` cada vez que se hace `ptr++`? Justifica la respuesta.
2. Modifica el código anterior para que los valores del vector `var` se incrementen en 1 utilizando los punteros `ptr`.

Ejercicio 3

Uno de los usos habituales de los punteros es la **memoria dinámica**. En particular, la función `malloc` permite solicitar, de forma dinámica, al sistema operativo el número de bytes que le hace falta a un proceso en un momento determinado.

```
int main () {  
  
    int i, *var;  
  
    var = malloc(1000 * sizeof(int));  
  
    for ( i = 0; i < 1000; i++) {  
        var[i] = 2 * i + 1;  
    }  
  
    free(var);  
  
    return 0;  
}
```

Ahora responde a las siguientes preguntas.

1. ¿Cuántos bytes se solicita al sistema operativo?
2. Modifica el código para que los valores del vector se inicialicen (a los valores que se ven en el código) utilizando un puntero `ptr` en lugar de inicializarlos utilizando `var[i]`.
3. ¿Por qué es más eficiente modificar los valores de la variable `var` utilizando punteros en lugar de acceder directamente utilizando `var[i]`? En otras palabras, ¿qué hace "internamente" el código hacer para acceder a `var[i]`?

Otras operaciones habituales son el paso de punteros a funciones. Esto permite que una función (por ejemplo, la función `main`) pase a otra función el valor de `var`. Dentro de esa función se pueden modificar los valores del vector. Estas modificaciones se podrán "ver" al devolver de la función.

```
void funcio(int *ptr)  
{  
    int i;  
    for(i = 0; i < 1000; i++)  
        ptr[i] = 2 * i + 1;  
}  
  
int main () {  
    int *var;  
  
    var = malloc(1000 * sizeof(int));  
    funcio(var);  
}
```

```
    free(var);  
  
    return 0;  
}
```

De la misma forma que se pueden pasar punteros a funciones, también se pueden devolver punteros de funciones.

Ejercicio 4

Un último tipo de puntero que queremos presentar son los **punteros sin tipo asociado** (int, float, double, ...). Estos tipos de punteros se llaman punteros genéricos.

```
int main () {  
    void *ptr;  
  
    ptr = malloc(4000);  
  
    // blah blah blah  
  
    free(ptr);  
  
    return 0;  
}
```

En este ejemplo sólo estamos indicando que queremos reservar 4000 bytes de memoria, pero no estamos diciendo (al declarar la variable) qué tipo de información (int, float, ...) almacenaremos en ella.

¿Cómo almacenar información? Ahora tocar manipular punteros.

```
int main () {  
    int *p_int;  
    void *ptr;  
  
    ptr = malloc(4000);  
  
    p_int = (int *) (ptr + 3);  
    *p_int = 50;  
  
    printf("Valor de ptr: %x\n", ptr);  
    printf("Valor de p_int: %x\n", p_int);  
  
    free(ptr);  
  
    return 0;  
}
```

Fíjate que en este código se declara un puntero de tipo genérico (`ptr`) así como uno de tipo entero (`p_int`). Después se realiza una operación aritmética con el puntero genérico, `ptr+3`, y se asigna al puntero de tipo entero mediante un *casting*.

1. ¿Qué estamos haciendo con esta operación de suma de punteros? ¿En cuándo se incrementa `ptr` al realizar la operación `ptr+3`?
2. ¿Dónde estamos almacenando el valor entero? Analiza bien los valores que se imprimen de `ptr` y `p_int`.

Ejercicio 5

Finalmente mostramos un ejemplo en el que se muestra una de las formas que se puede utilizar para **reservar memoria dinámica para una matriz**.

```
int main(void)
{
    char a[10][10]; // matriu de 10x10 de char

    a[0][0] = 0;
    a[0][0] = a[0][0] + 1;

    return 0;
}
```

Reservar memoria de forma estática tiene sus inconvenientes. En la siguiente sesión de problemas veremos que el hecho de hacerlo estáticamente no permite reservar matrices de tamaño muy grande (más de 10MBytes, por ejemplo). Es más conveniente reservar la memoria de forma dinámica.

En lenguaje C, reservar memoria para una matriz de forma dinámica, implica un procedimiento algo tedioso. La primera vez que se hace puede parecer complejo. Una vez entendido se puede sacar provecho a la hora de implementar determinadas funcionalidades. A continuación, se muestra cómo reservar memoria dinámica para una matriz de caracteres de tamaño 10x10 (es el equivalente al ejemplo anterior).

```
int main(void)
{
    int i;
    char **a; // matriu de 10x10 de char

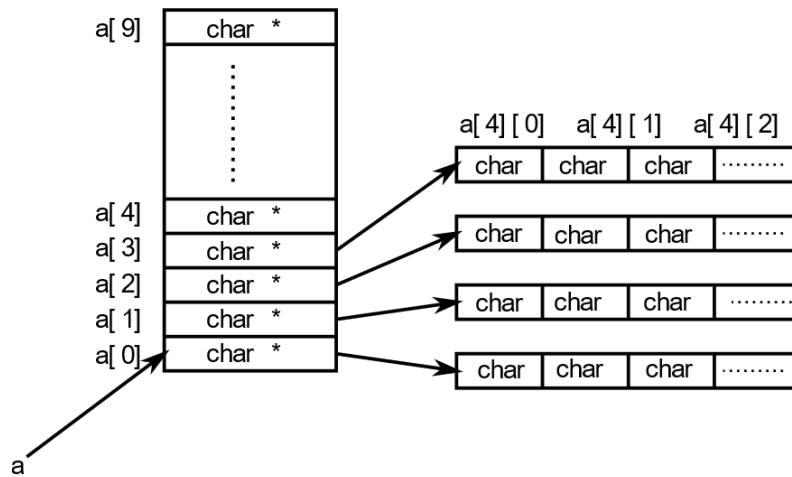
    a = (char **) malloc(sizeof(char *) * 10); // Pas 1
    for(i = 0; i < 10; i++) // Pas 2
        a[i] = (char *) malloc(sizeof(char) * 10); // Pas 2

    a[0][0] = 0;
    a[0][0] = a[0][0] + 1;

    for(i = 0; i < 10; i++) // Pas 3
        free(a[i]); // Pas 3
    free(a); // Pas 4

    return 0;
}
```

A continuación se muestra un esquema de lo que se hace en memoria al reservar memoria para la matriz.



Responder a las siguientes preguntas.

1. ¿Qué es lo que se hace en el paso 1? ¿Y al paso 2?
2. Observar cómo se libera la memoria. ¿Se puede dar el paso 4 antes que el paso 3?

Ten en cuenta que aquí se ha mostrado una forma de reservar memoria de forma dinámica en la matriz. Hay otras formas de hacerlo y todo depende del contexto y la aplicación en la que se utilizará la matriz.